ЭФФЕКТИВНЫЕ НЕЙРОСЕТИ · ДОМАШНЯЯ РАБОТА №2

Super-Convergence

Very Fast Training of Neural Networks Using Large Learning Rates

Leslie N. Smith · Nicholay Topin · arXiv:1708.07120v3, 2018

Разбор статьи и воспроизведение ключевых экспериментов: MNIST/LeNet, CIFAR-10/ResNet-20 и ResNet-56, LR range test, секущая оценка шага, абляции.

Перенос протокола Caffe на PyTorch, 26 обучающих запусков, NVIDIA H100. Все отчёты, артефакты и код в этом репозитории.

ГЛАВНЫЙ РЕЗУЛЬТАТ СТАТЬИ (FIG. 1A)

92,4 %

за 10 000 обновлений с одним циклом LR

91,2 %

за 80 000 обновлений у piecewise-constant baseline

Восемь раз меньше обновлений на ResNet-56 с CIFAR-10.

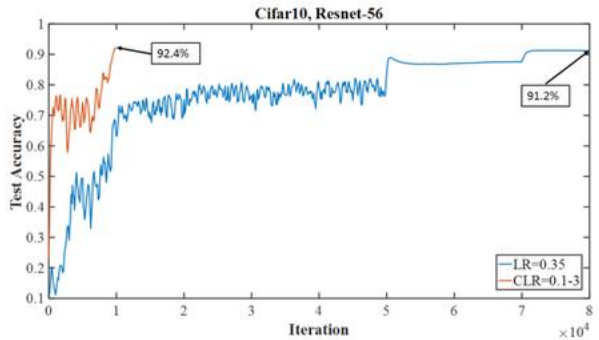
Это сравнение двух конкретных рецептов, а не гарантия ускорения для любой сети. Дальше в презентации: что из этого подтвердилось в моих запусках.

Можно ли научиться быстрее за счёт очень большого LR

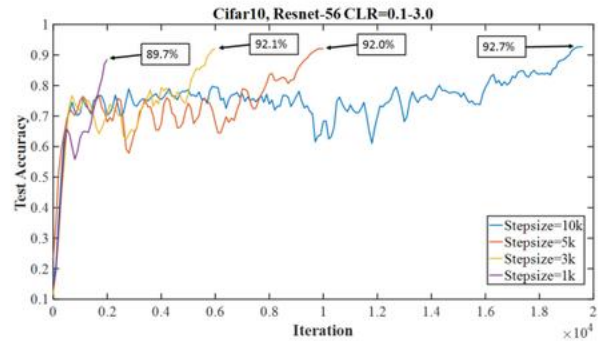
Обычный протокол CIFAR: постоянный LR 0,35, два понижения в 10 раз (на 50k и 70k обновлений), всего 80 000 обновлений.

Авторы предлагают: один линейный цикл, который разгоняет LR до 3,0 и обратно. Качество получается выше, а итераций в разы меньше. Эффект назван супер-сходимостью.

Почему это странно: шаг 3,0 при batch 1000 на порядок больше обычных значений. Без batch norm и правильного balance регуляризации такое обучение расходится.



(a) Comparison of test accuracies of super-convergence example to a typical (piecewise constant) training regime.



(b) Comparison of test accuracies of super-convergence for a range of stepsizes.

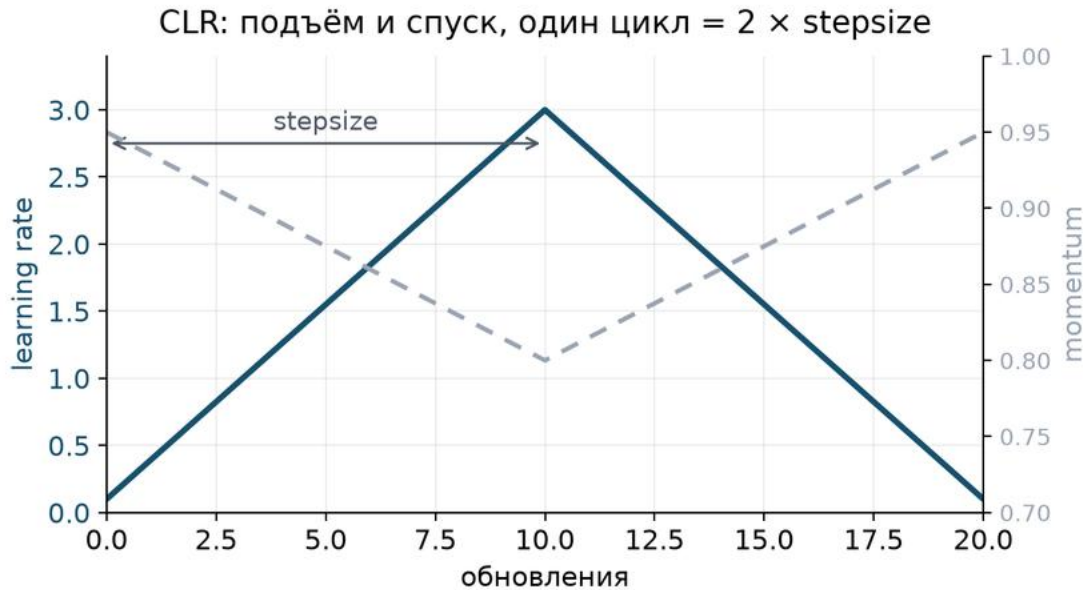
Figure 1 статьи: слева CLR против PC до 80k итераций, справа разные stepsize. Взято из 1708.07120v3.

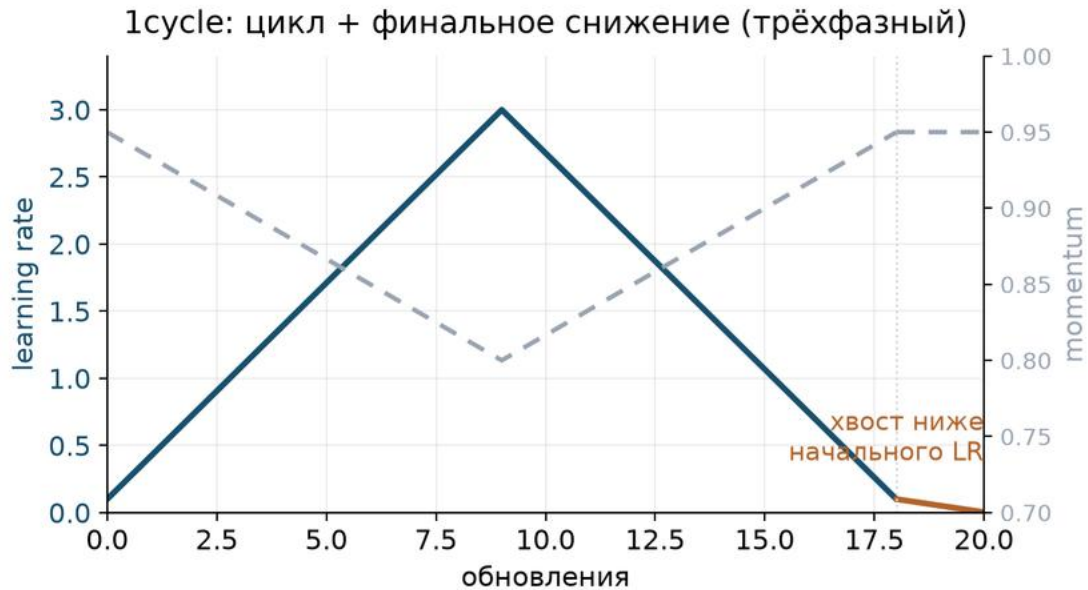
92,4 % CLR · 10 000 обновлений

91,2 % PC · 80 000 обновлений

8 × отношение бюджетов (не доказательство ускорения)

Два разных рецепта: треугольный CLR и трёхфазный 1cycle





CLR

Линейный подъём от min_lr до max_lr за stepsize обновлений и такой же спуск. Длина одного цикла: 2 × stepsize. Сравнения CIFAR в статье (Fig. 4b) используют именно его.

1cycle

Более поздний рецепт: цикл короче всего обучения, затем хвост, который опускает LR ниже начального значения. MNIST-эксперимент статьи (таблица 2) построен на нём.

Momentum в противофазе

Растёт LR, падает momentum: 0,95 → 0,8 → 0,95. Идея авторов: при маленьком LR нужен сильный momentum, чтобы усреднять шум градиентов.

Большой LR работает как регуляризатор, и его надо балансировать

WEIGHT DECAY

1e-4

При 1e-3 супер-сходимость ломается: сеть почти не различает классы в середине цикла. 1e-4 рабочий уровень (Fig. 12b).

BATCH NORM

MAF 0,95

За 10k обновлений стандартная бегущая статистика (0,999) не успевает перестроиться. Авторы снижают MAF до 0,95; длина цикла и MAF подбираются вместе (Table 6).

BATCH SIZE

1000

92,4 % при batch 1000 против 89,5 % при 256 и 92,1 % при 1536 (Table 7). Больший batch в этом рецепте помогает.

MOMENTUM

0,9

Лучший измеренный: 92,4 % против 90,7 % при 0,95 (Table 8). Универсальный оптимум из этого не следует.

Как это объясняют авторы (Appendix A): большой шаг выводит обучение в плоскую область loss (иллюстрация Goodfellow et al.), плюс шум градиентов на больших LR действует как регуляризация. Это интерпретация, а не измеренное свойство сети.

Границы: с Adam эффекта получить не удалось (Fig. 9); для bottleneck-блоков устойчивый LR заметно меньше 3 (Fig. 13). Большие LR не универсальны.

LR range test находит область, формула (8) её объясняет

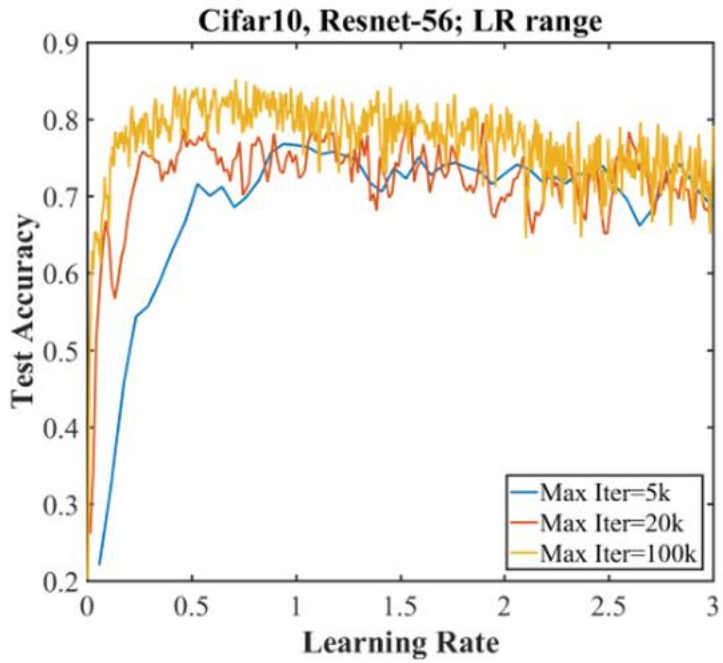


Figure 2b статьи: range test на ResNet-56, ramps 5k/20k/100k итераций.

- **Range test:** линейно растить LR в течение короткого прогона и смотреть на loss/accuracy. Точность не падает вплоть до 3,0: сеть допускает шаги на порядок больше обычных.
- **Практика статьи:** range test идёт перед супер-сходящимся обучением и задаёт max_lr.

Секущая оценка шага (формула 8):

```
eta* = eta · Σ|theta[i+1] − theta[i]| / Σ|2·theta[i+1]
− theta[i] − theta[i+2]|
```

- Три последовательных вектора весов дают оценку кривизны без обращения Hessian; поверх применяется EMA (alpha 0,1).
- **Для ResNet-56 авторы получают 2-6** уже за первые 300 итераций: большие LR теоретически оправданы, кривизна вдоль спуска мала.
- Оценка приближённая; полная версия статьи оставляет адаптивный optimizer на будущее.

Что я воспроизводил и в какой постановке

В объёме

- **MNIST / LeNet:** inverse policy (85 эпох) против трёхфазного 1cycle (12 эпох), 3 seed + контроль с постоянным momentum.
- **CIFAR-10 / ResNet-20:** CLR 20k против PC 80k, 3 seed. Бюджеты с Fig. 4b.
- **CIFAR-10 / ResNet-56:** центральная пара статьи: CLR 10k против PC 80k, seed 0.
- **Диагностика:** LR range test (LeNet, ResNet-56), секущая оценка LR, расписания и loss.
- **Абляции:** weight decay, пик LR, накопление BN, объём данных (10k/25k), 20k обновлений, seed 0.

Вне объёма

- ImageNet, CIFAR-100, Wide-ResNet, DenseNet, ResNeXt, dropout-свитчи.
- Много-GPU BN, как у авторов (batch 125 × 8 GPU); у меня BN считается на полном batch 1000 на одном GPU.
- Повторные seed для ResNet-56: пара PC/CLR в один запуск.

ПРОТОКОЛ СРАВНЕНИЙ

Метрика: accuracy последнего checkpoint после полного бюджета на всех 10 000 test-изображений. Test не участвует в подборе. Семена 0/1/2, deterministic cuDNN. Среднее ± выборочное std, не доверительный интервал. Окружение: NVIDIA H100, PyTorch 2.14, порт протокола Caffe.

Чтобы сравнивать расписания, нужно перенести протокол

Блок ResNet по Table 3 и prototxt авторов:



shortcut идёт прямо в Sum: лишней ReLU перед сложением нет

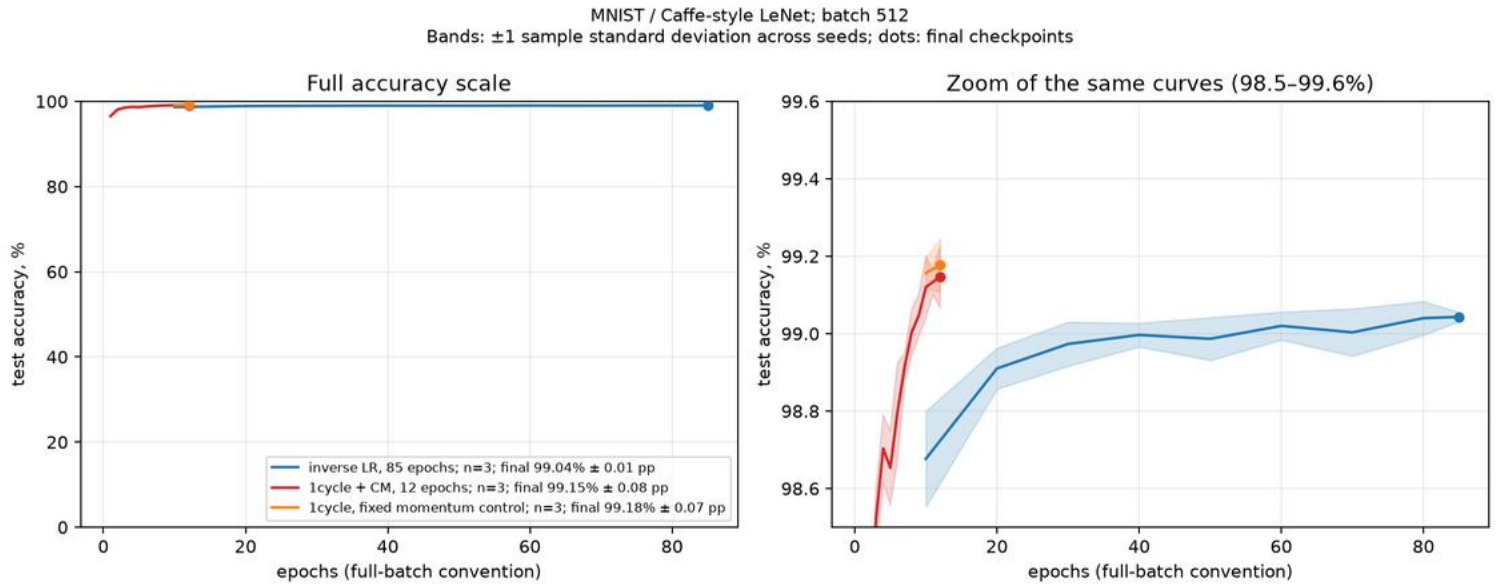
Downsampling (Table 4): свёртки сохраняют старую ширину, stride 2 у первой; shortcut через average pooling 3×3 без padding; сначала Sum и ReLU, затем дописываются нулевые каналы.

Первая свёртка stride 1: в Table 5 PDF стоит stride 2, но это противоречит финальному average pooling 8×8; исполняемый prototxt авторов подтверждает stride 1.

- **BatchNorm по Caffe:** накопительные mean/var и нормализующий множитель; MAF 0,95 = мой bn-momentum 0,05 (вес нового наблюдения).
- **SGD:** закон momentum Caffe h = μ·h + LR·g пересчитан под переменный LR; у PyTorch история масштабируется иначе.
- **Bias и decay:** lr_mult 2 для bias, у conv bias decay нет; инициализация MSRA.
- **Данные:** пиксели 0-255, вычитание mean image, затем horizontal flip (как Caffe DataTransformer); только полные batch: 50 upd/epoch CIFAR, 117 MNIST.
- **Проверка:** check_solution.py покрывает блоки, BN, границы расписаний, LR-множители и порядок шагов momentum.

Главная адаптация: BN на полном batch 1000 на одном GPU вместо ~125 на восьми. Это может влиять на устойчивость, и я это не изолировал.

12 эпох против 85: воспроизводится по форме, чуть ниже по числу



Мои запуски, 3 seed, финальные checkpoint; слева полная шкала, справа зум 98,5-99,6 %.

INVERSE, 85 ЭПОХ

99,04 ± 0,01

у авторов: 99,03 ± 0,04

1CYCLE + CM, 12 ЭПОХ

99,15 ± 0,08

у авторов: 99,25 ± 0,03

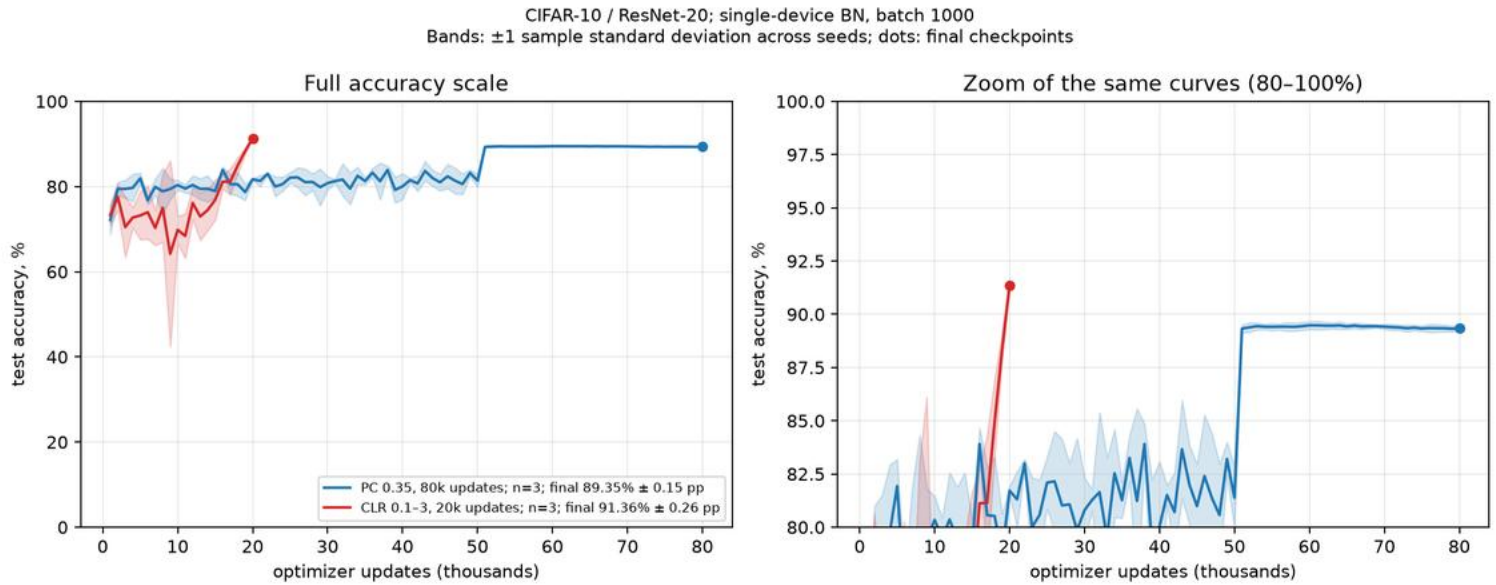
1CYCLE, MOM CONST

99,18 ± 0,07

контроль: momentum 0,9

Базовая линия сошлась почти дословно с таблицей 2. 1cycle отстаёт от 99,25 примерно на 0,1 п.п. при 7× меньшем бюджете (1 404 обновления против 9 945). Постоянный momentum не хуже цикла 0,95 → 0,8.

Главное сравнение: CLR выше PC при четырёхкратно меньшем бюджете



Полные траектории test accuracy, 3 seed, полосы ± выборочное std; точки финальные checkpoint.

CLR · 20 000 ОБНОВЛЕНИЙ

91,36 ± 0,26 %

LR 0,1 → 3 → 0,1, stepsize 10k

PC · 80 000 ОБНОВЛЕНИЙ

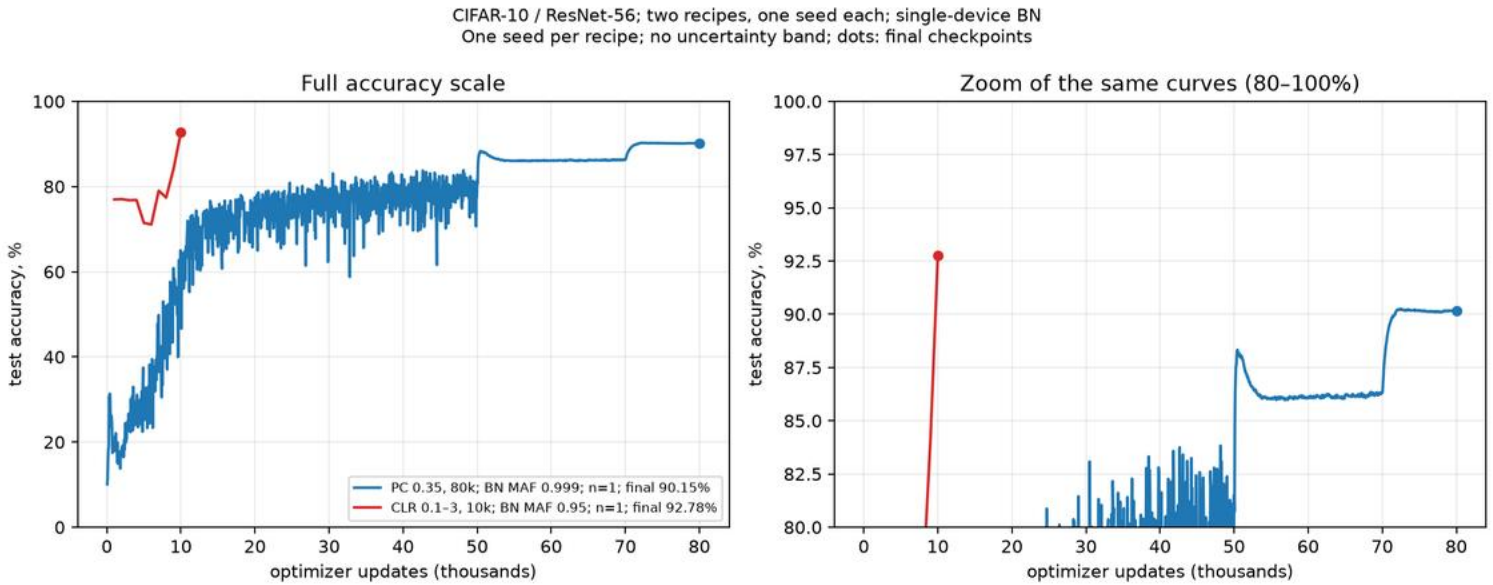
89,35 ± 0,15 %

LR 0,35, drops ×0,1 на 50k/70k

+2,01 п.п. при 4× меньшем числе обновлений; разрыв больше разброса seed. Часть отставания PC - недообученный baseline: его train растёт вплоть до 80k.

Время (1212 с против 4938 с) - не бенчмарк: общий сервер, параллельные процессы.

Центральная пара статьи: 10k против 80k обновлений



CLR · 10 000

92,78 %

у авторов: 92,4 %

PC · 80 000

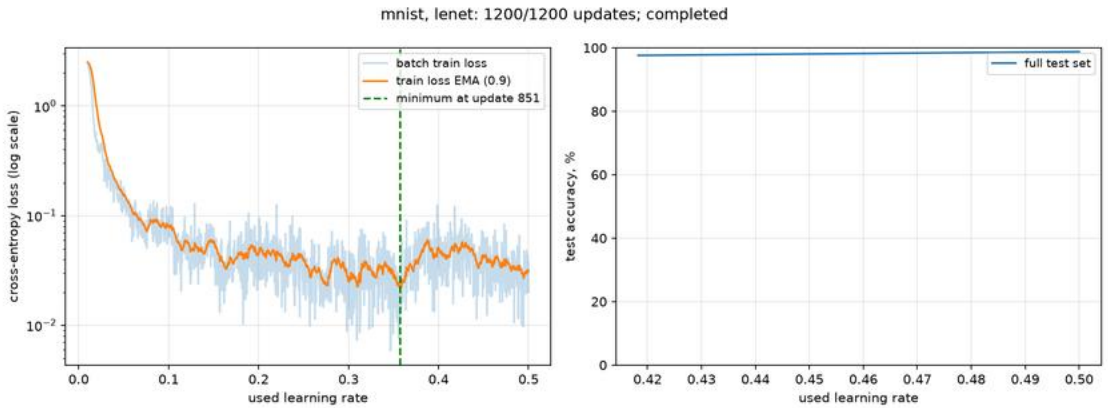
90,15 %

у авторов: 91,2 %

CLR-плечо даже выше статьи, PC ниже на 1 п.п. Плечи различаются не только LR, но и BN MAF (рецепт против рецепта). Один seed: сравниваю числа, а не распределения.

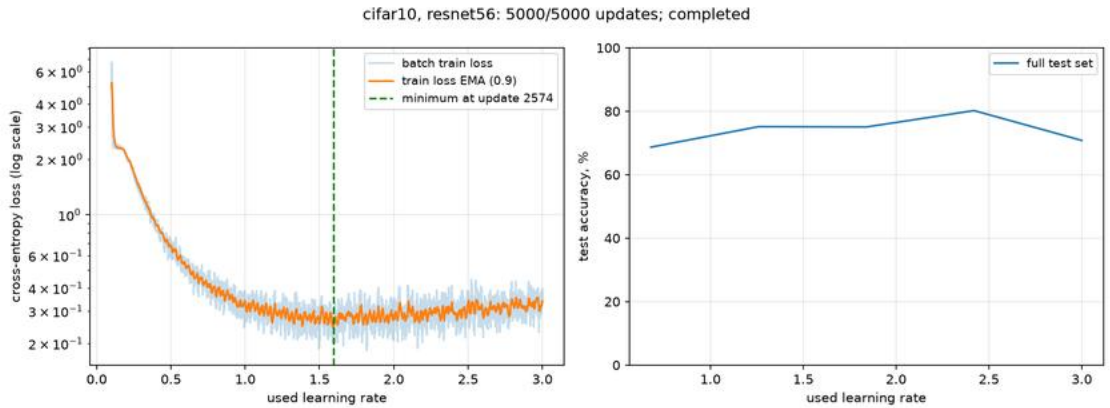
ResNet-56, seed 0. CLR 10k (MAF 0,95) против PC 80k (MAF 0,999), бюджеты как в Fig. 1a.

LR range test: оба прогона дошли до конца



range_lenet · 1200/1200 шагов, completed

Минимум EMA train loss: обновление 851 при LR 0,357.

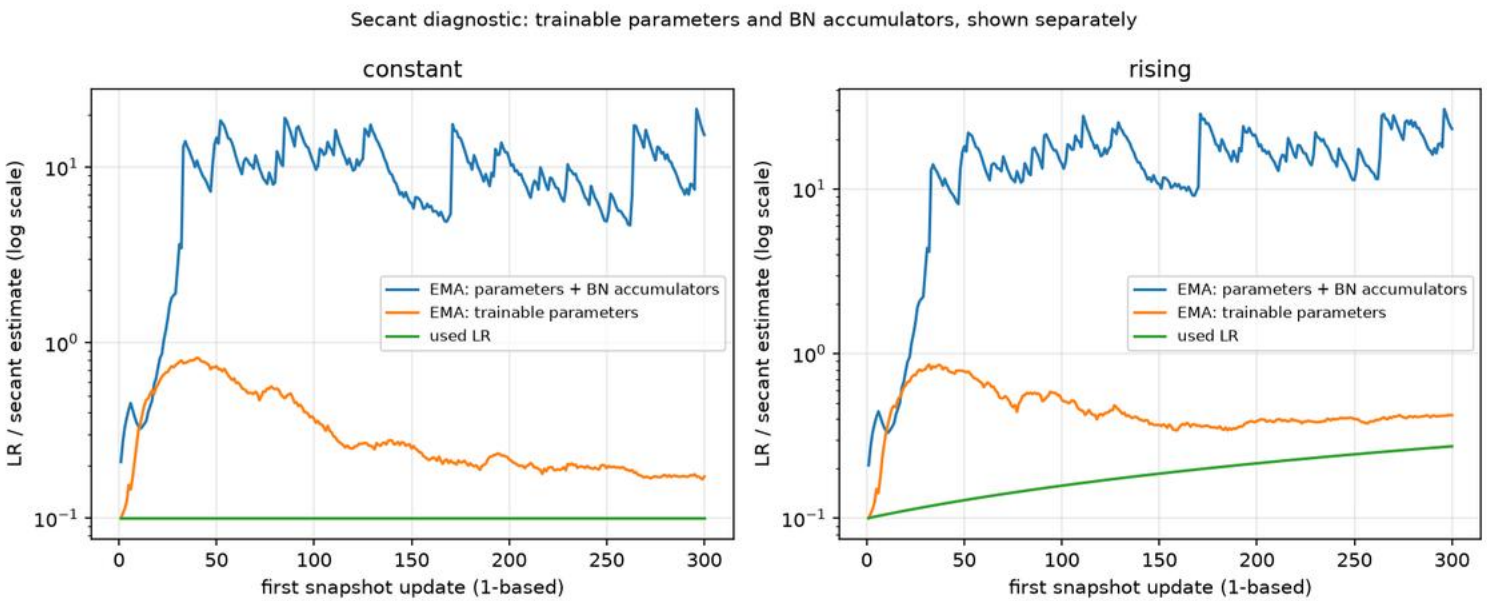


range_resnet56 · 5000/5000 шагов, completed

Минимум EMA train loss: обновление 2574 при LR 1,593.

Критерий - train loss, не test accuracy. Бюджеты основных запусков взяты из рецептов статьи; правило «10 × индекс минимума» в v3 отсутствует и у меня не использовалось.

Секущая оценка LR: форма есть, диапазон 2-6 отсутствует



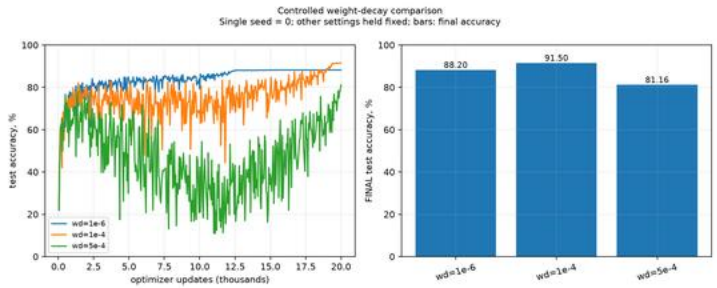
Оценка (8) с EMA 0,1, первые 300 обновлений: constant LR 0,1 и rising LR на ResNet-56.

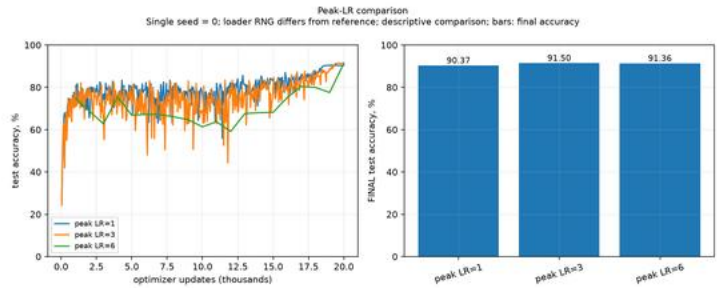
режим	параметры	параметры + BN-статистики
constant	0,173	15,3
rising	0,423	23,2

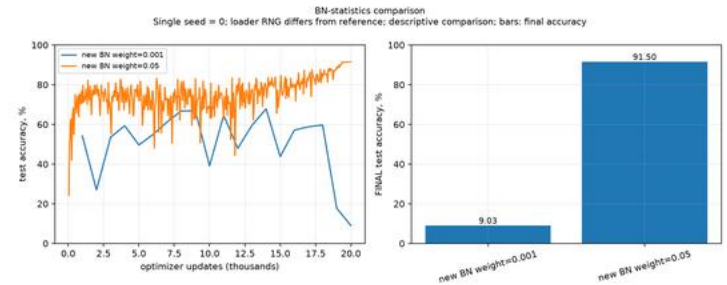
Авторский L2TermComputation3.py читает net.params целиком, включая BN-накопители. Их включение меняет масштаб на порядок, и это не переменные gradient descent.

Итог: качественно повторил Fig. 3a (постоянный LR оседает к своему значению, rising растёт вместе с LR), но не 2-6. Оценку называю индикатором «можно ещё поднять», независимого подтверждения кривизны у меня нет.

Weight decay, пик LR и BN при 20k обновлений (seed 0)







WEIGHT DECAY

1e-6: 88,20 % (−3,2 п.п. к baseline), 5e-4: 81,16 % (−10,2). Баланс регуляризации из статьи подтверждается направленно.

ПИК LR

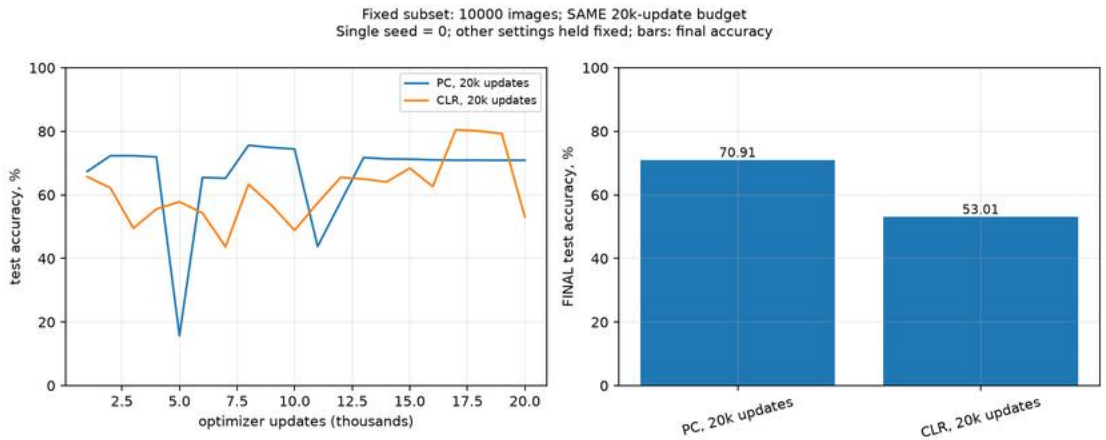
Пик 6 держится на уровне пика 3 (91,36), пик 1: 90,37. Верхняя граница цикла не критична; «нечувствительность LR» по одному seed не утверждаю.

СКОРОСТЬ ОБНОВЛЕНИЯ BN

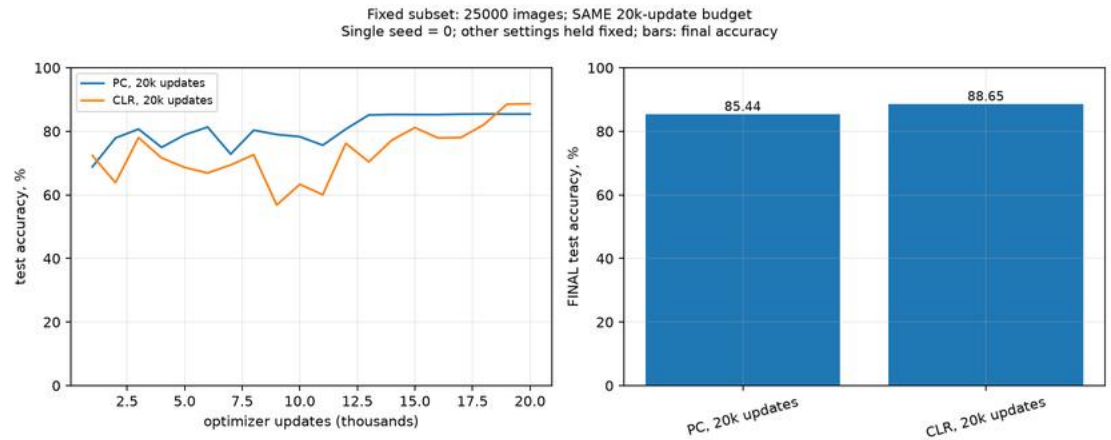
MAF 0,999 в коротком CLR-цикле ломает обучение: 9,03 %. Быстрые BN-статистики действительно нужны, как в статье.

Один seed; для пика LR 6 и BN 0,001 менялся и загрузчик данных - эти две строки описательные.

Выигрыш CLR на малых выборках подтвердился частично



10 000 изображений: CLR 53,01 % против PC 70,91 %. Направление статьи здесь не воспроизвелось.



25 000 изображений: CLR 88,65 % против PC 85,44 %. Здесь CLR впереди, как и ждали.

Постановка отличается от Table 1: у меня ResNet-20 и равные бюджеты 20k в каждой паре, у авторов ResNet-56 и 10k против 80k. Один seed; mean image вычтен по всем 50k train-изображений.

Что подтвердилось, а что нет

ПОДТВЕРЖДЕНО

- CLR стабильно выше PC на ResNet-20: +2,01 п.п. при 4× меньшем бюджете (3 seed).
- ResNet-56 на уровне статьи: 92,78 % за 10k обновлений (seed 0).
- MNIST 1cycle: 99,15-99,18 % за 12 эпох против 99,04 % за 85.
- Big-LR-обучение требует быстрого BN и малого weight decay - направления абляций совпадают со статьёй.

РАСХОЖДЕНИЯ

- Secant-оценка даёт 0,2-0,4 (и до 23 с BN-накопителями) вместо диапазона 2-6; форма кривой при этом как на Fig. 3a.
- На 10k изображений CLR проиграл PC с равным бюджетом (53,0 против 70,9).
- PC-плечо ResNet-56 ниже авторского: 90,15 против 91,2.

ГРАНИЦЫ

- BN на полном batch 1000 на одном GPU вместо 125 × 8 - не изолирован.
- Абляции в один seed, времена не бенчмарк, отношение бюджетов не равно ускорению.
- ImageNet, CIFAR-100 и другие архитектуры не запускались.

Итог: супер-сходность как режим «один цикл большого LR + правильные BN и decay» воспроизводится в PyTorch; количественные части (оценка шага, выигрыш на 10k) остаются не подтверждёнными.

Всё воспроизводимо из репозитория

Отчёт	report.md + report.pdf: протокол, таблицы, все оговорки. Полный разбор статьи: paper_analysis.md
Код	code/train.py, policies.py, resnet.py, lenet.py, range_test.py; проверки code/check_solution.py, code/verify_report2.py
Запуски	runs/: 28 папок, history.json (args, SHA256, статус), lr_log.json, iter_log.json, сохранённые модели
Проверка чисел	figures/verification.json: accuracy каждого финального checkpoint пересчитана по всем 10 000 test-изображений

Воспроизвести с нуля:

```
python code/check_solution.py · bash code/run_all.sh all --gpus 0 1 · python code/verify_report2.py
```